Project Report

TITLE: MOODFLIX - MOOD-BASED MOVIE RECOMMENDATION SYSTEM

Mini Project of ML

Subject Code: 25CAP-702

Submitted By:
Name: Sneha Mittal
UID: 25MCA20004
Class: MCA
Section: 3/A

Submitted To:
Mr. Munish Kumar
Assistant Professor

Masters in Computer Applications (MCA)
University Institute of Computing
Chandigarh University,
Gharuan, Mohali, Punjab, India-140413

TABLE OF CONTENTS

1. Declaration

2. Acknowledgement

3. Abstract

4. Introduction

5. Hardware and Software Requirements

6. Methodology

7. Flowchart

8. Implementation (Code and Output)

9. Conclusion and Future Scope

10. References

DECLARATION

I hereby declare that the project titled “MoodFlix - Mood-Based Movie Recommendation System” is an academic mini project prepared under the guidance of Mr. Munish Kumar for the subject Machine Learning (Subject Code: 25CAP-702).

This report describes a Streamlit application that recommends movies from a manually entered catalog using mood, genre, industry, rating, and keyword preferences. Its ranking method uses assigned metadata and a simple scoring rule; it does not train or load a machine-learning model.

The work presented in this report has not been submitted, either in part or in full, to any other institution for the award of a degree or diploma.

Student’s Name: Sneha Mittal
UID: 25MCA20004

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty guide, Mr. Munish Kumar, for his guidance and support during the development of my mini project titled “MoodFlix - Mood-Based Movie Recommendation System.” His guidance helped me understand how a small interactive application can organize catalog information and respond to user preferences.

I am thankful to Chandigarh University and the University Institute of Computing for providing an academic environment and learning resources that supported this project. I also appreciate the Python and Streamlit documentation that helped me build the application interface. Finally, I thank my family and friends for their encouragement and support.

Sneha Mittal
UID: 25MCA20004

ABSTRACT

MoodFlix is a Streamlit-based movie discovery application that helps a user browse a manually curated catalog according to the kind of movie they want to watch. The current source code contains 51 movie records from Bollywood, Hollywood, and international cinema. Each record includes a title, year, genres, mood tags, rating, short description, search tags, industry, and a display color.

The user can choose a mood, industry, genre, minimum rating, keyword, and number of recommendations. The application filters records against those preferences and sorts the matching movies with a transparent scoring rule. The base score is the movie rating, with optional bonuses for a matching genre and a keyword match. The mood match is also represented in the score, although the mood filter already restricts the candidate set to that mood.

The interface presents movie cards and a session-only watchlist. Catalog data is embedded directly in the Python source; the project does not require a database, an external movie API, or an API key. The current version is a rule-based recommendation demonstration, not a trained machine-learning system.

Features:

- Mood-based filtering across 12 mood choices.
- Filtering by industry, genre, rating threshold, and keyword.
- Sorted movie recommendations with selectable result counts of 3, 6, 9, or 12.
- A session-only watchlist for saving titles while the app is open.
- A dark, cinematic interface with movie cards and ratings.

INTRODUCTION

Choosing a movie can take time when a viewer has a feeling or occasion in mind but no specific title. MoodFlix offers a lightweight way to explore familiar titles by describing the desired mood and narrowing the list with additional preferences.

The application is implemented in Python with Streamlit. Movie information and recommendation tags are stored as Python records in app.py. When the page is rendered, the selected filters are applied to the catalog. The matching records are then sorted by a score built from the supplied rating and optional preference bonuses.

Objectives

- To build an interactive movie discovery interface using Python and Streamlit.
- To organize a small catalog with mood, genre, industry, rating, and keyword metadata.
- To let users combine multiple filters to find relevant titles.
- To rank matching movies with a clear and understandable scoring rule.
- To provide a temporary watchlist during the active application session.
- To document the current implementation accurately and identify future machine-learning extensions.

Scope and limitation

Recommendations are limited to the titles and tags written into the source file. There is no trained model, user-history learning, online catalog lookup, or permanent watchlist storage. Results therefore depend on the completeness and consistency of the manually assigned metadata.

HARDWARE AND SOFTWARE REQUIREMENTS

Software requirements

- Programming language: Python.
- Web application framework: Streamlit.
- Interface styling: Streamlit widgets with custom CSS.
- Project files: app.py, requirements.txt, and README.md.
- Browser: a modern web browser to view the local Streamlit application.
- Operating system: Windows, macOS, or Linux with Python and package installation available.

Hardware requirements

- A computer capable of running Python and a local Streamlit process.
- Keyboard and pointing device for selecting preferences and managing the watchlist.
- Memory and storage sufficient for Python and the Streamlit dependency; the catalog itself is small and stored in source code.

  The project does not need a GPU, database server, trained model file, network API, or API key to retrieve movie data. The interface imports Google Fonts through its CSS when an internet connection is available; the application logic and embedded movie catalog remain local.

METHODOLOGY

1. Catalog definition: Movie records are defined in app.py. Each record stores a title, release year, genres, moods, rating, emoji, card color, description, search tags, and industry.

2. Preference collection: Streamlit controls collect a mood, an industry, a genre, an optional keyword, a minimum rating, and a result count.

3. Candidate filtering: A movie must match the selected mood and any chosen industry, genre, rating threshold, and keyword. The keyword is checked against the title, description, genres, tags, and industry.

4. Scoring: Each candidate starts with its catalog rating. A mood match adds 2 points; a selected genre match adds 1.5 points; and a matching keyword in the title, description, or tags adds 2 points.

5. Ranking and display: Candidates are sorted from highest to lowest score. The interface shows the match count and up to the selected number of movie cards.

6. Watchlist handling: Selecting a card button adds or removes that title in Streamlit session state. The list is temporary and is cleared when the session is reset.

Because every candidate must already pass the selected mood filter, the mood bonus is constant within a result set. With no genre or keyword preference, the order is therefore determined by the manually supplied rating. The method is deterministic and does not learn from previous users or viewing activity.

FLOWCHART

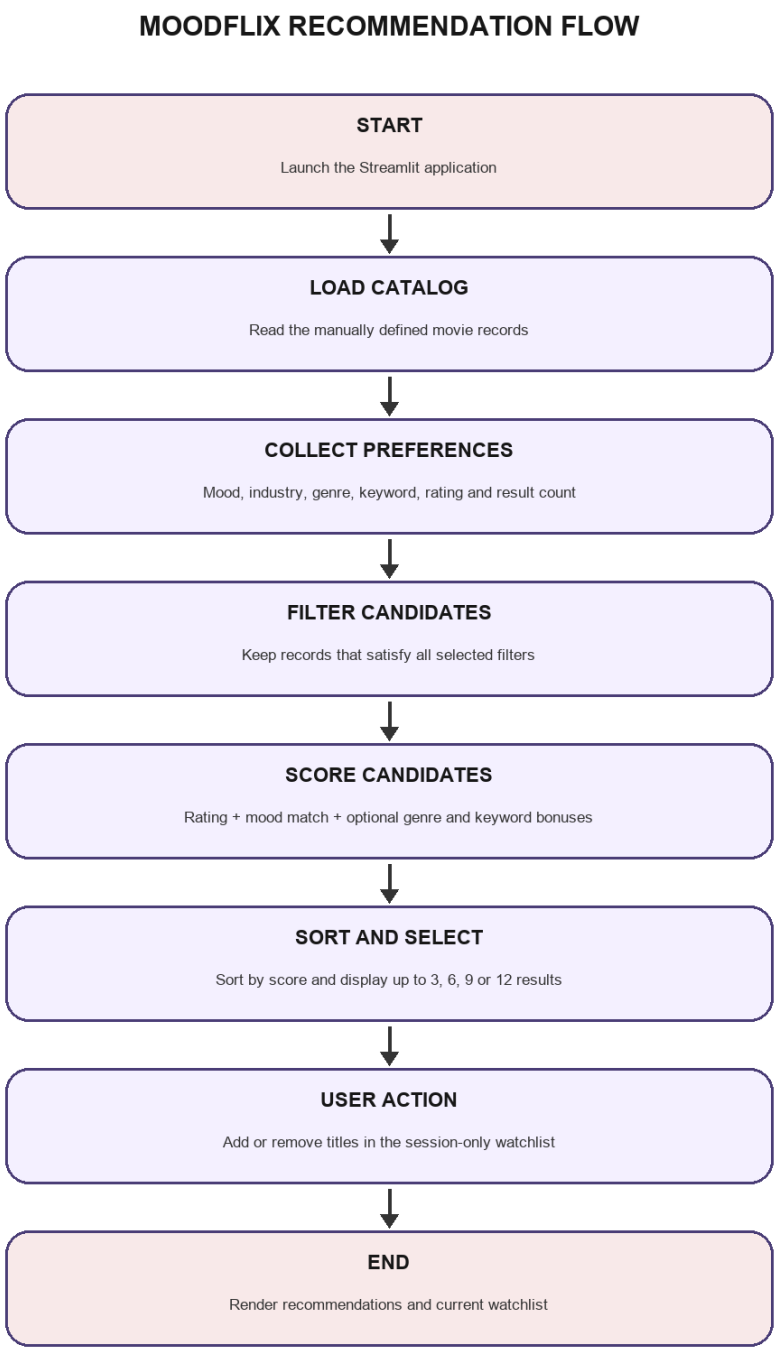


Figure 1. Recommendation and watchlist flow in the current MoodFlix application.

IMPLEMENTATION (CODE AND OUTPUT)

The project is intentionally small. The catalog, filters, scoring logic, and interface are kept in app.py, while requirements.txt lists Streamlit as the runtime dependency. README.md explains how to install the dependency and start the application.

Recommendation score

The current scoring function can be summarized as follows:

```
score = rating
       + 2.0 if selected mood matches
       + 1.5 if selected genre matches
       + 2.0 if keyword occurs in title, description, or tags
```

Before scoring, the application filters candidates by mood and any selected industry, genre, rating threshold, and keyword. It then sorts the remaining records by score in descending order and limits the displayed cards to the selected count.

Project files and run command

```
MoodFlix_Manual_Movies/
  app.py           # catalog, filtering, ranking, and interface
  requirements.txt   # Streamlit dependency
  README.md         # setup and feature notes

python -m pip install -r requirements.txt
python -m streamlit run app.py
```

The watchlist is stored in st.session_state and is not written to a file or database. Movie ratings and metadata are manually supplied catalog values.

IMPLEMENTATION OUTPUT

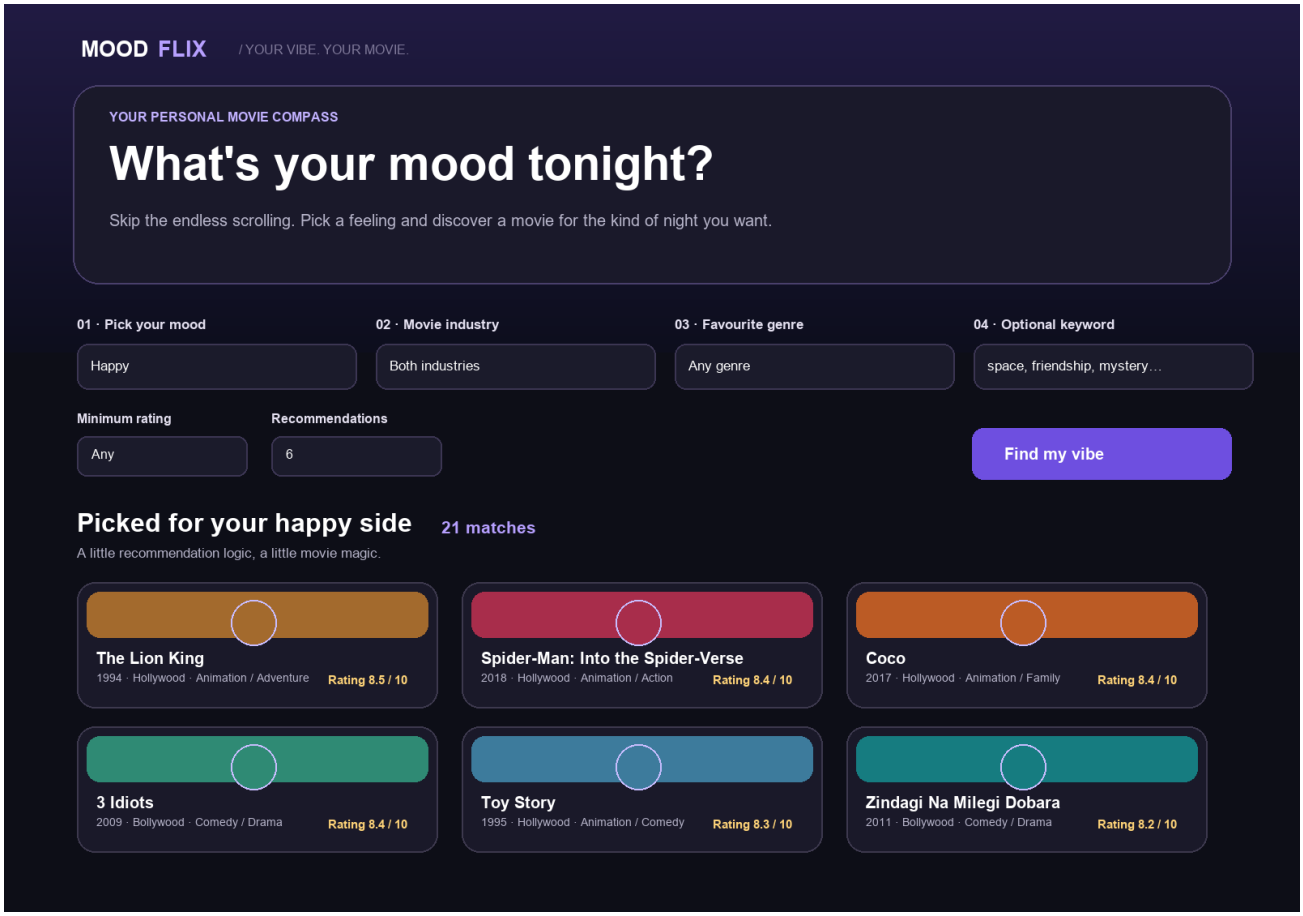


Figure 2. Representative interface output based on the app layout and catalog. Example selections: Happy mood, both industries, any genre, no keyword, any rating, and six displayed recommendations.

With these selections, the current catalog produces 21 Happy-mood matches. The interface displays the top six in descending rating order; equal ratings retain the source catalog order. This example illustrates the filter and ranking behavior, not an output from a trained model.

CATALOG SUMMARY

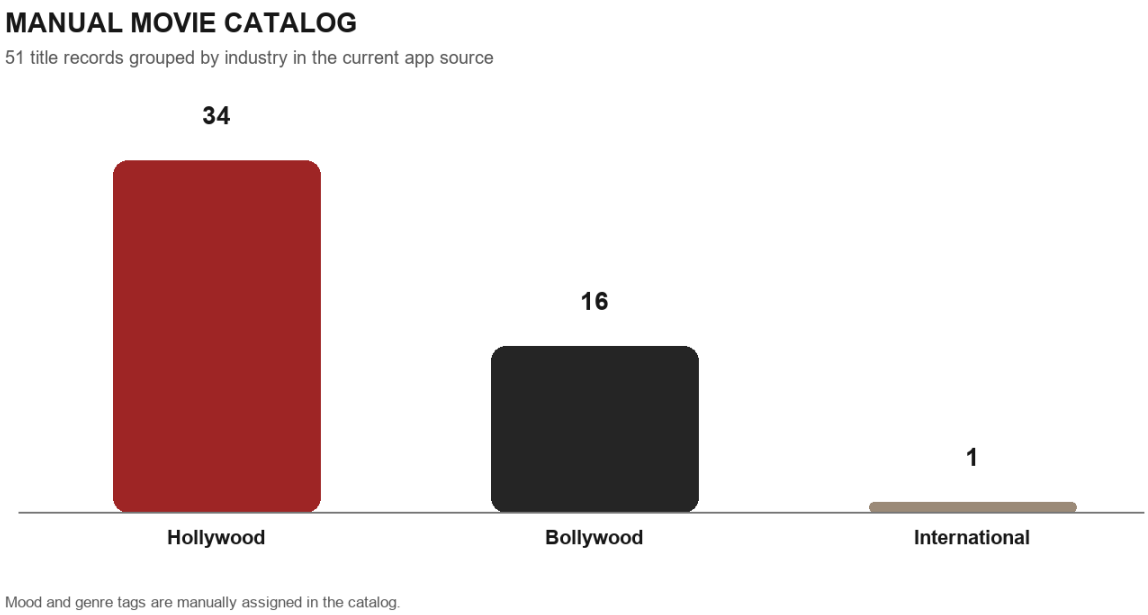


Figure 3. Movie records by industry, counted from the current app.py catalog.

The current source contains 51 entries: 34 Hollywood titles, 16 Bollywood titles, and 1 International title. The catalog uses 12 moods and 17 distinct genre labels. These counts describe the code as inspected for this report and may change when the catalog is edited.

The output cards display title, year, industry, up to two genre labels, description, rating, and genre tags. The mood and filter state determine which records appear; the card styling is part of the Streamlit interface.

CONCLUSION

MoodFlix demonstrates a simple way to browse a curated movie list through mood and preference filters. The Streamlit interface combines a manually tagged catalog, filtering, a transparent score, sorted results, and a temporary watchlist. The implementation is lightweight and does not depend on a database or a remote movie service.

The recommendation method is rule-based. It does not train a machine-learning model or infer new preferences from user behavior. The quality of its results depends on the manually assigned moods, genres, keywords, and ratings. For a Machine Learning extension, a labeled movie dataset and a defined relevance target would be needed before training and evaluating a recommendation model.

Future scope

- Expand and validate the catalog and its mood, genre, and keyword tags.
- Add movie descriptions or user-selected favorites to support richer content-based matching.
- Explore TF-IDF and cosine similarity only after preparing a suitable text dataset and evaluation plan.
- Add tests for filter combinations, keyword matching, score ordering, and empty results.
- Store watchlists persistently if users need them across sessions.
- Improve accessibility, responsive layout, and clearer explanations for why each title was recommended.

REFERENCES

- MoodFlix project source code, app.py. Manual catalog, mood and genre definitions, filter logic, scoring, card rendering, and session watchlist.
- MoodFlix project README.md. Setup commands and feature notes.
- MoodFlix project requirements.txt. Streamlit dependency declaration.
- Python Documentation. https://docs.python.org/3/
- Streamlit Documentation. https://docs.streamlit.io/
- Streamlit API reference for session state. https://docs.streamlit.io/develop/api-reference/caching-and-state/st.session_state